

08 Geometric Transformations

8.1 Geometric Transformations and Matrix Multiplication

Scaling, rotation, shear, orthogonal projection, reflection: revisiting five planar transforms through matrix multiplication



Linear Algebra Made Easy and Visual | Open-source code & drafts: <https://github.com/Visualize-ML>

Core Skills You Will Master

1

Linear mapping: understand how matrix multiplication $\mathbf{Ax}=\mathbf{y}$ maps an input vector $\mathbf{x}$ to an output vector $\mathbf{y}$

3

Geometric meaning of special column vectors: the zero vector $\mathbf{0}$, unit vectors $\mathbf{e}_1$, $\mathbf{e}_2$, and the all-ones vector $\mathbf{1}$

5

Scaling maps to a diagonal matrix; rotation maps to an orthogonal matrix with determinant 1

7

Orthogonal projection reduces dimension, determinant 0, matrix not invertible

2

Under a linear transform, the unit square grid becomes a parallelogram grid: parallel and equidistant, with the origin fixed

4

Review transpose of a matrix product: $(\mathbf{AB})^T=\mathbf{B}^T\mathbf{A}^T$, and what it means for composing transforms

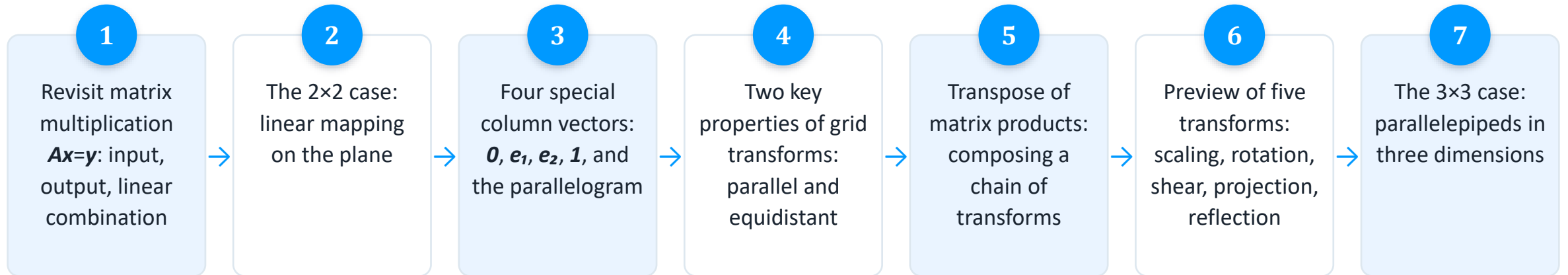
6

Shear maps to an upper/lower triangular matrix (diagonal entries 1), determinant 1

8

Reflection maps to an orthogonal matrix with determinant -1

From Matrix Multiplication to Five Transforms



Seven Lenses for Every Transform in This Chapter

① Matrix-multiplication view: how the matrix acts on $\mathbf{0}$, $\mathbf{e}_1$, $\mathbf{e}_2$, and the all-ones vector $\mathbf{1}$

② The determinant: is $|\det|$ greater than 1? Positive, negative, or zero? Determinants of special matrices

③ How the determinant reflects the area or volume change caused by the transform

④ The inverse matrix (does it exist?) and the inverse transform; inverses of special matrices

⑤ Linear-combination view of the parallelogram/parallelepiped grid, and whether dimension drops

⑥ Basis view: does the transform matrix form a basis, and what kind of basis

⑦ Properties of the transform matrix: symmetric, diagonal, orthogonal, invertible, commutative

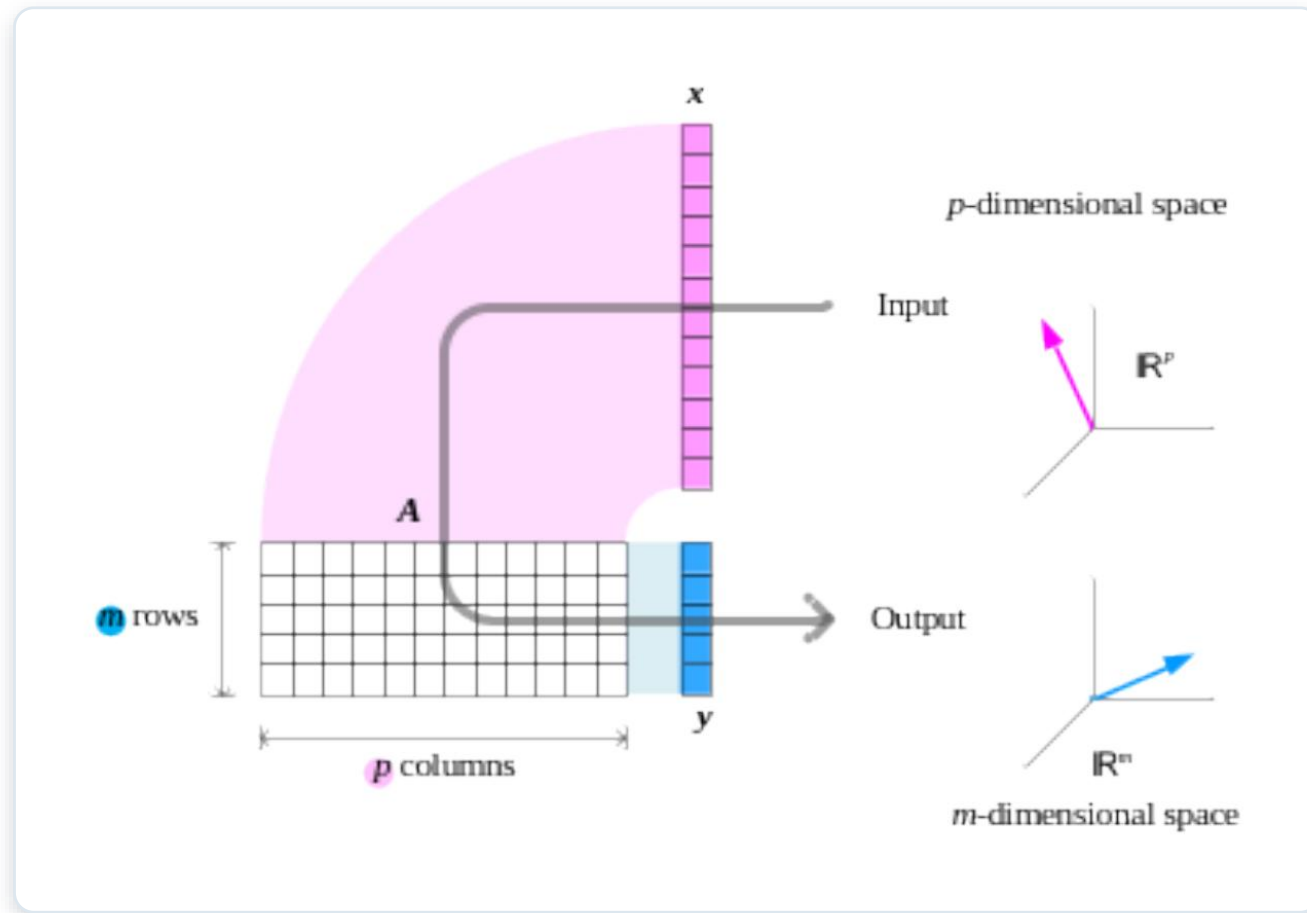
Matrix Multiplication $Ax=y$: Input and Output

$$Ax = y$$

- Matrix multiplication is a linear mapping: matrix A maps a p -dimensional input x to an m -dimensional output y
- In $(m \times p) @ (p \times 1)$, the shared p is “cancelled”, leaving shape $(m \times 1)$; y is a linear combination of A 's columns

Fig 1 · Input and Output of $Ax=y$

- Note matrix A here is a wide (flat) matrix, so its columns cannot form a basis



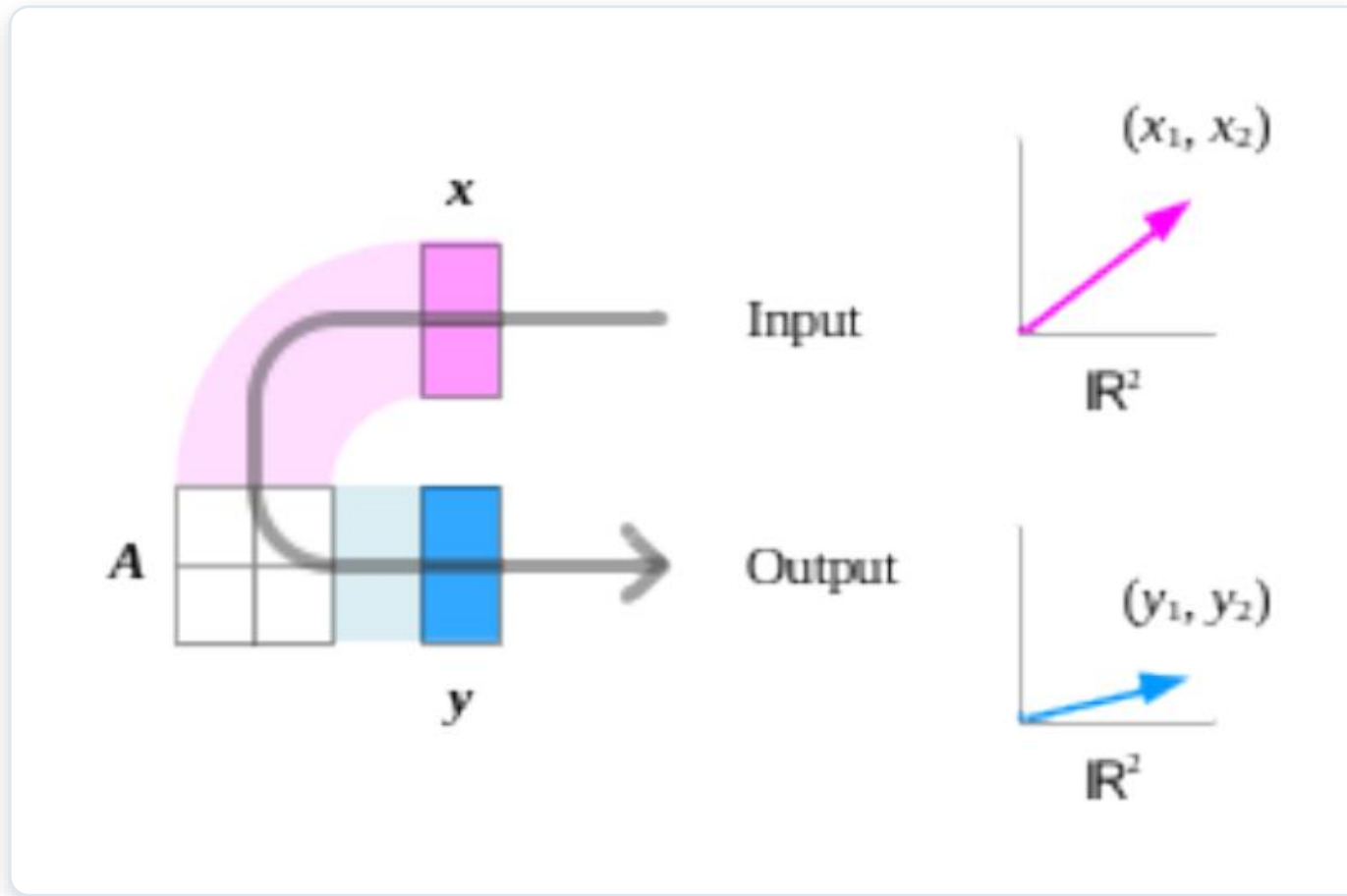
Expanding $Ax=y$ from the Third Perspective

$$Ax = x_1 \mathbf{a}_1 + x_2 \mathbf{a}_2 = \mathbf{y}$$

- Input $\mathbf{x}$ is a 2×1 vector, a point (x_1, x_2) on the plane; output $\mathbf{y}$ is also 2×1, a point (y_1, y_2)
- $\mathbf{y}$ is a linear combination of $\mathbf{a}_1$ and $\mathbf{a}_2$ — this is matrix multiplication's “third perspective”

Fig 2 · $Ax=y$, Matrix A Is 2×2

- Both input x and output y can be seen as vectors starting at the origin



MATRIX MULTIPLICATION ON FOUR SPECIAL VECTORS

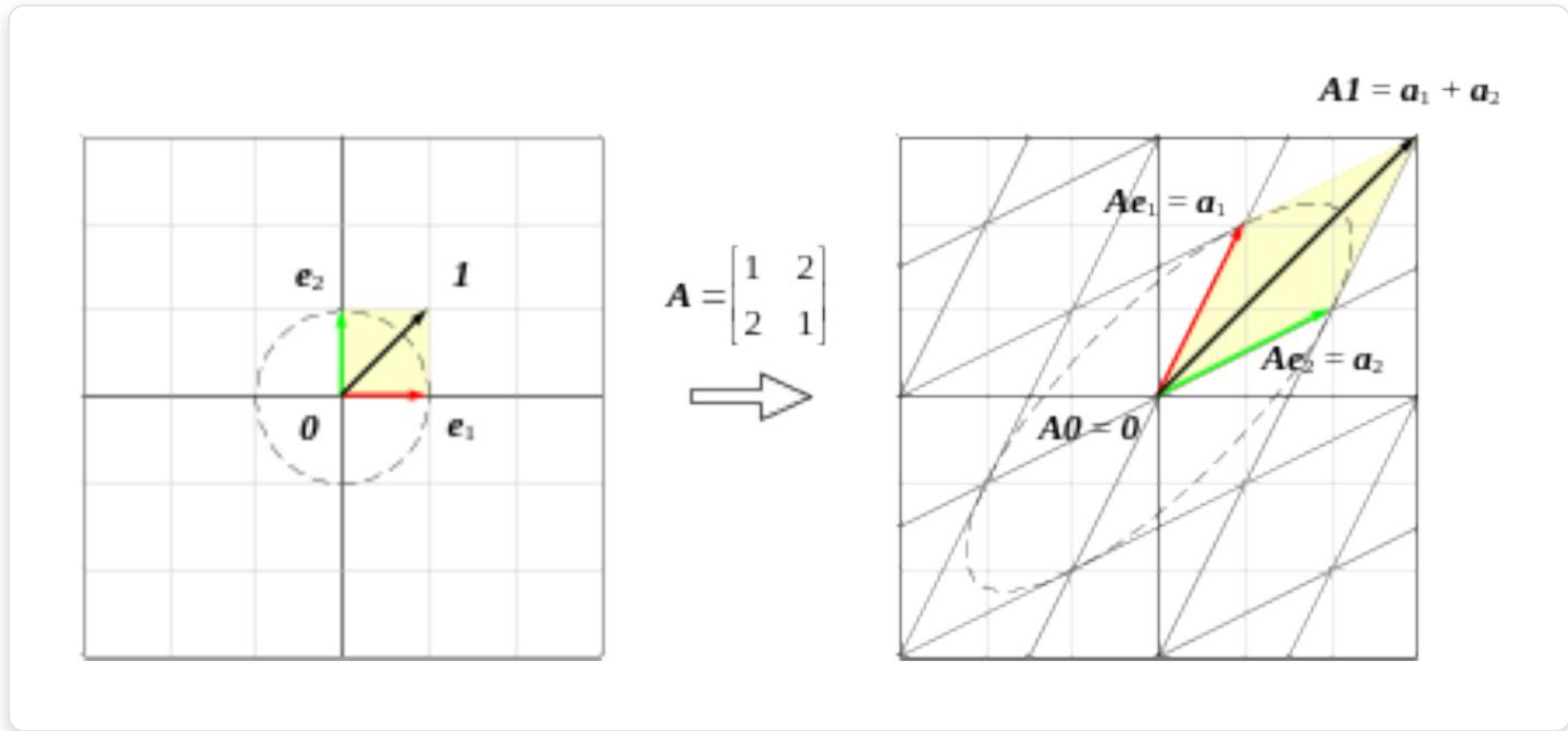
$$\mathbf{A}\mathbf{0}=\mathbf{0}, \mathbf{A}\mathbf{e}_1=\mathbf{a}_1, \mathbf{A}\mathbf{e}_2=\mathbf{a}_2, \mathbf{A}\mathbf{1}=\mathbf{a}_1+\mathbf{a}_2$$

$$\mathbf{A}\mathbf{0}=\mathbf{0} \quad \mathbf{A}\mathbf{e}_1=\mathbf{a}_1 \quad \mathbf{A}\mathbf{e}_2=\mathbf{a}_2 \quad \mathbf{A}\mathbf{1}=\mathbf{a}_1+\mathbf{a}_2$$

- The zero vector $\mathbf{0}$ stays $\mathbf{0}$ after the transform — the origin never moves, a key property of linear transforms
- $\mathbf{A}\mathbf{e}_1$ and $\mathbf{A}\mathbf{e}_2$ extract $\mathbf{A}$'s first and second columns; $\mathbf{A}\mathbf{1}$ is the sum of $\mathbf{A}$'s two columns

Fig 3 · Four Special Column Vectors on the Plane

- The unit square (pale yellow) becomes a parallelogram; $\det(\mathbf{A})=-3$, area 3 — the sign shows $\mathbf{a}_1$ trails $\mathbf{a}_2$ (counterclockwise)



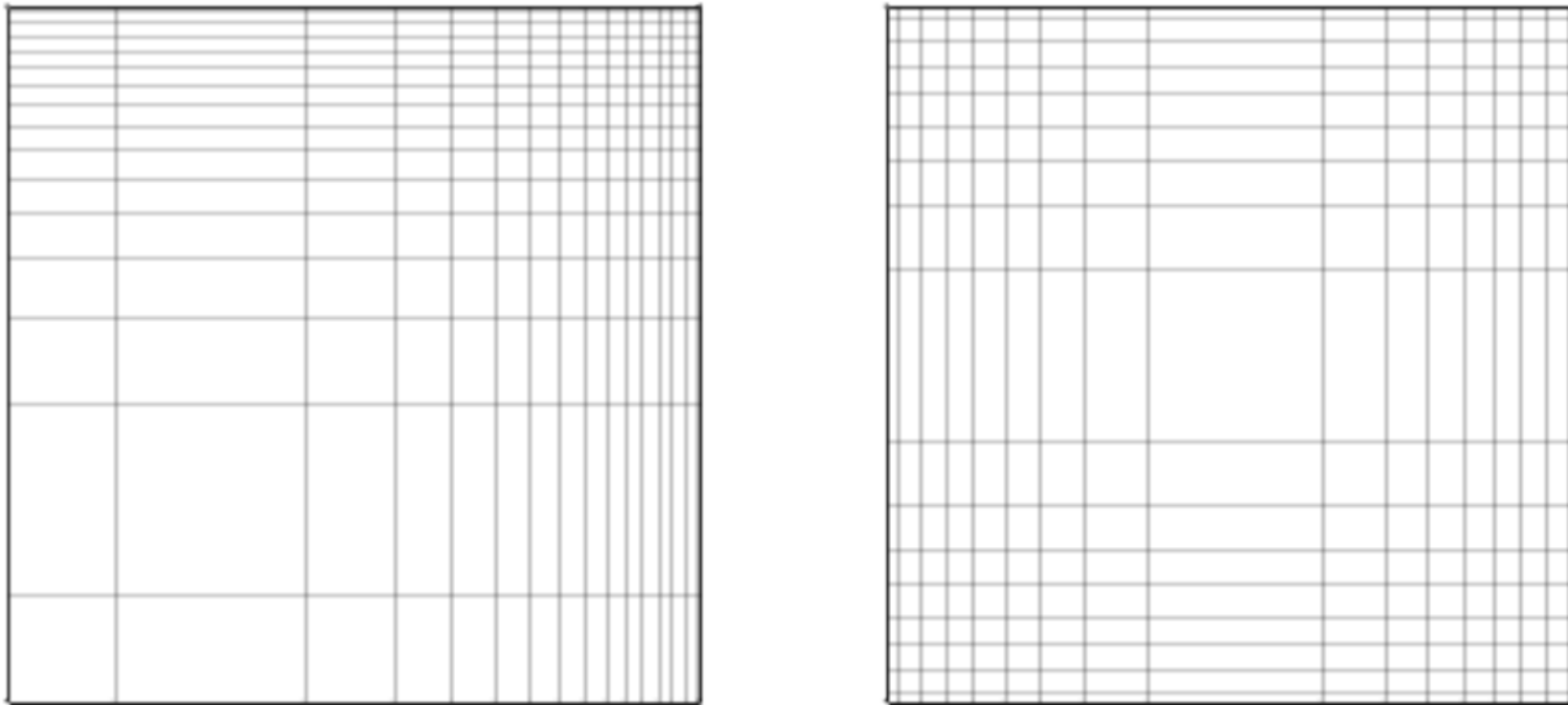
The Grid Becomes a Parallelogram Grid: Parallel and Equidistant

$$k_1 \mathbf{e}_1 + k_2 \mathbf{e}_2 \rightarrow k_1 \mathbf{a}_1 + k_2 \mathbf{a}_2$$

- Before: the unit-square grid spans the standard basis $[\mathbf{e}_1, \mathbf{e}_2]$; after: if $\mathbf{a}_1, \mathbf{a}_2$ are independent, the grid spans basis $[\mathbf{a}_1, \mathbf{a}_2]$
- “Parallel” alone is not enough — “equidistant” is the real test for whether a transform is linear

Fig 4 · Nonlinear Transforms

- Both grids stay parallel after the transform, but are no longer equidistant — so neither transform here is linear



Composing a Chain of Geometric Transforms

$$(A\mathbf{x})^T = \mathbf{x}^T A^T; \quad DCBA\mathbf{x}$$

- Transposing gives a row vector describing the same point; $(AB)^T = B^T A^T$ reverses the order in a chain product
- $DCBA\mathbf{x}$ applies a chain of transforms to $\mathbf{x}$ right-to-left: $A \rightarrow B \rightarrow C \rightarrow D$

Scaling, Rotation, Shear, Orthogonal Projection, Reflection

Scaling Stretches or compresses a shape proportionally; the transform matrix is diagonal

Rotation Preserves length and angle; the transform matrix is orthogonal, determinant 1

Shear Tilts a shape along one direction; the matrix is upper/lower triangular (diagonal entries 1), determinant 1

Orthog. Projection Projects a shape onto a subspace, reducing dimension; determinant 0, matrix not invertible

Reflection Flips a shape about a line through the origin; the matrix is orthogonal, determinant -1

Fig 5 · Matrix A Is 2×2 , Scaling

- The unit-square grid becomes a square/rectangle grid; when $\mathbf{a}_1, \mathbf{a}_2$ are independent, basis $[\mathbf{a}_1, \mathbf{a}_2]$ is orthogonal

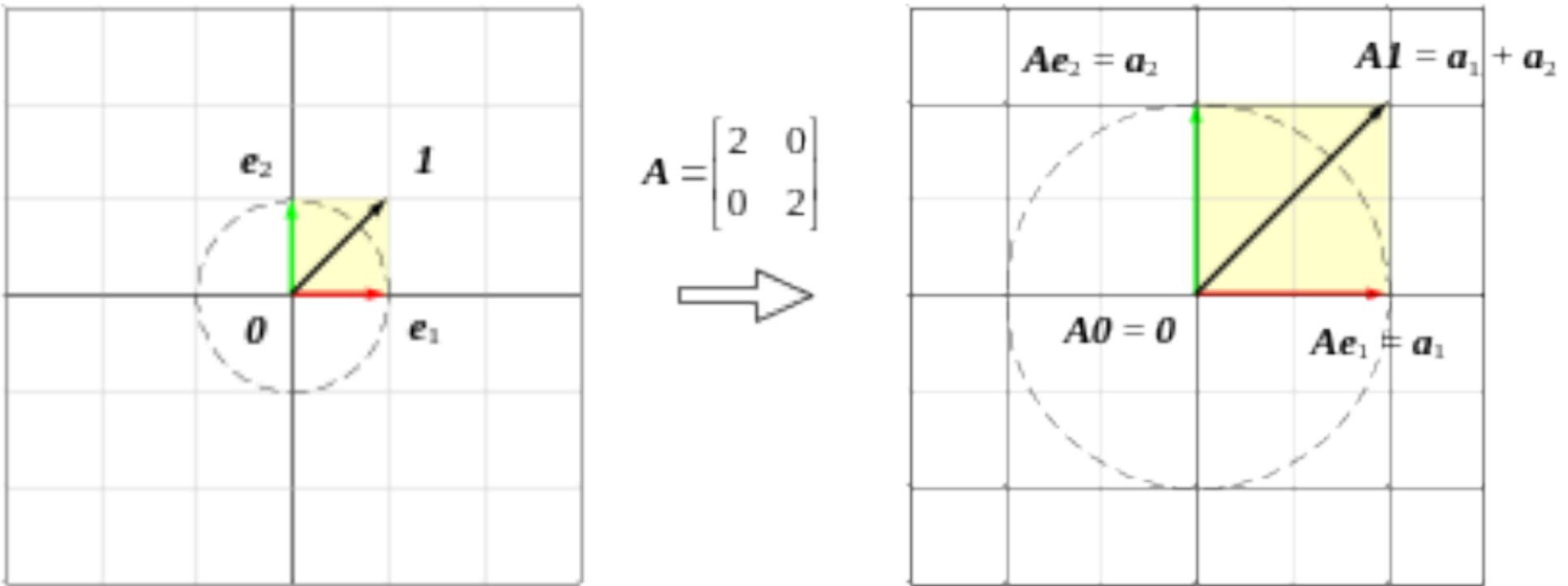


Fig 6 · Matrix A Is 2×2 , Rotation About the Origin

- The axis-aligned unit-square grid (standard basis) becomes a rotated unit-square grid; $AA^T = A^T A = I$

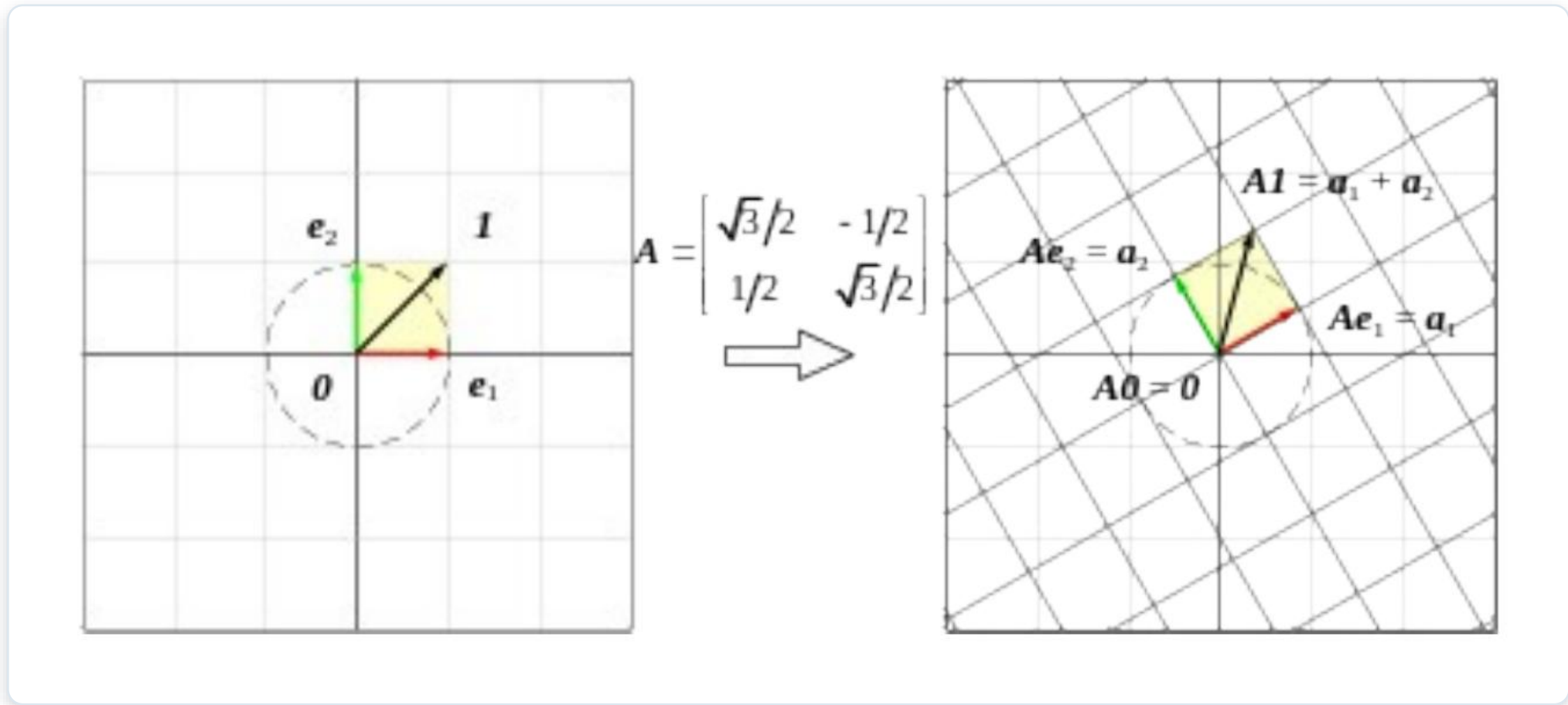


Fig 7 · Matrix A Is 2×2 , Horizontal Shear

- $\mathbf{a}_1$ is a horizontal unit vector; the unit circle becomes an ellipse — a topic revisited later in LDL decomposition

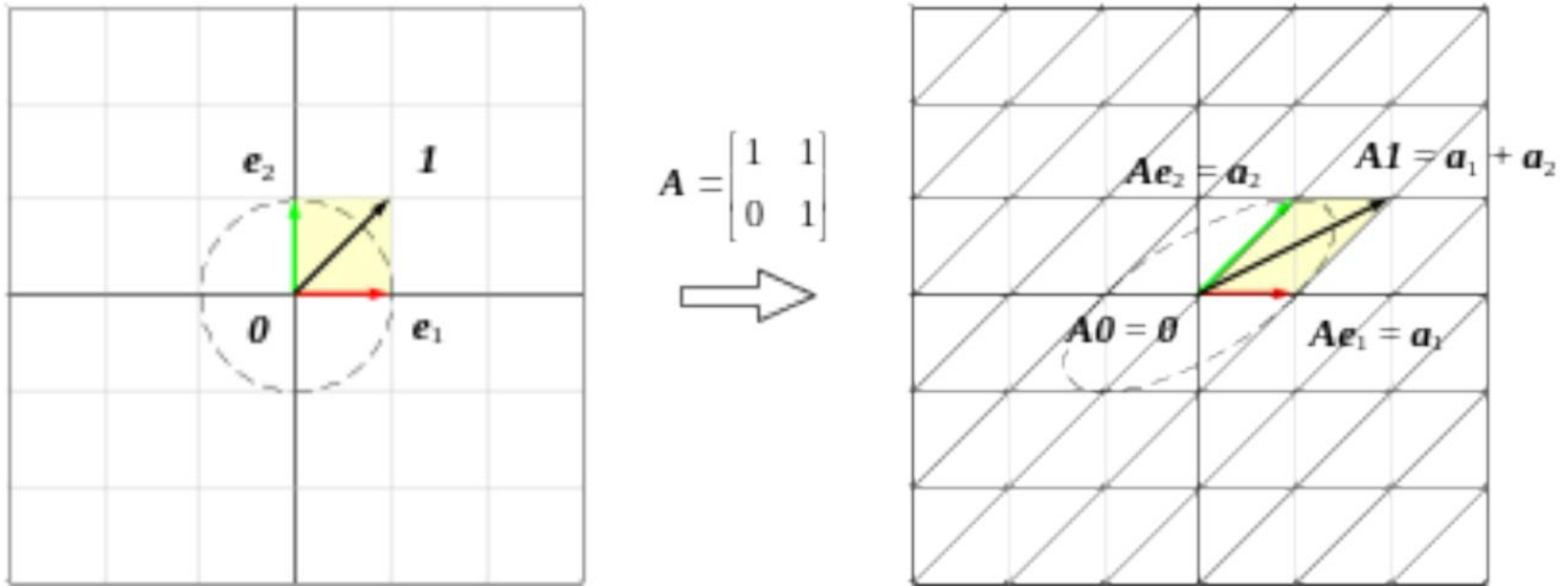


Fig 8 · Matrix A Is 2×2 , Orthogonal Projection

- $A\mathbf{1} = \mathbf{a}_1 + \mathbf{a}_2 = \mathbf{0}$: the columns are dependent and cannot form a basis; determinant 0, matrix not invertible

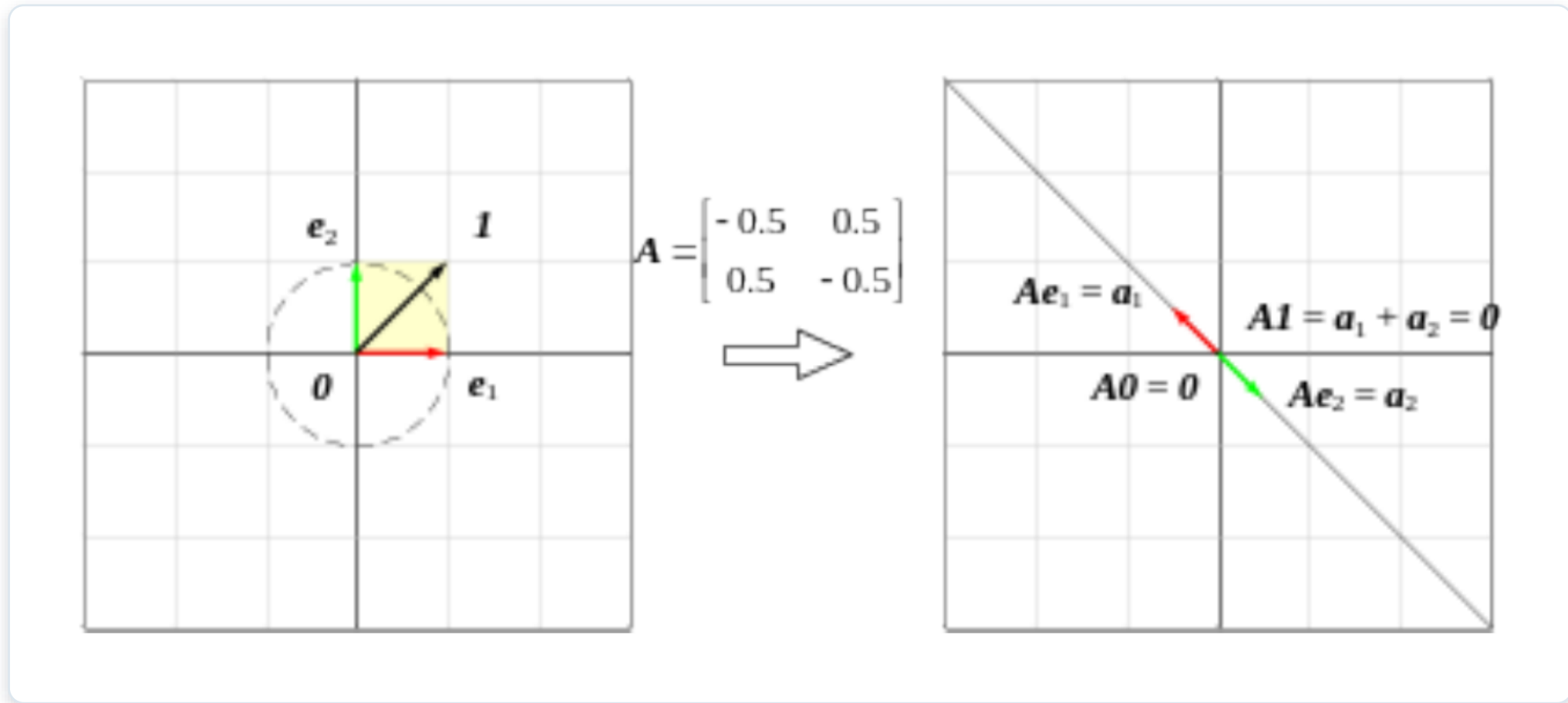
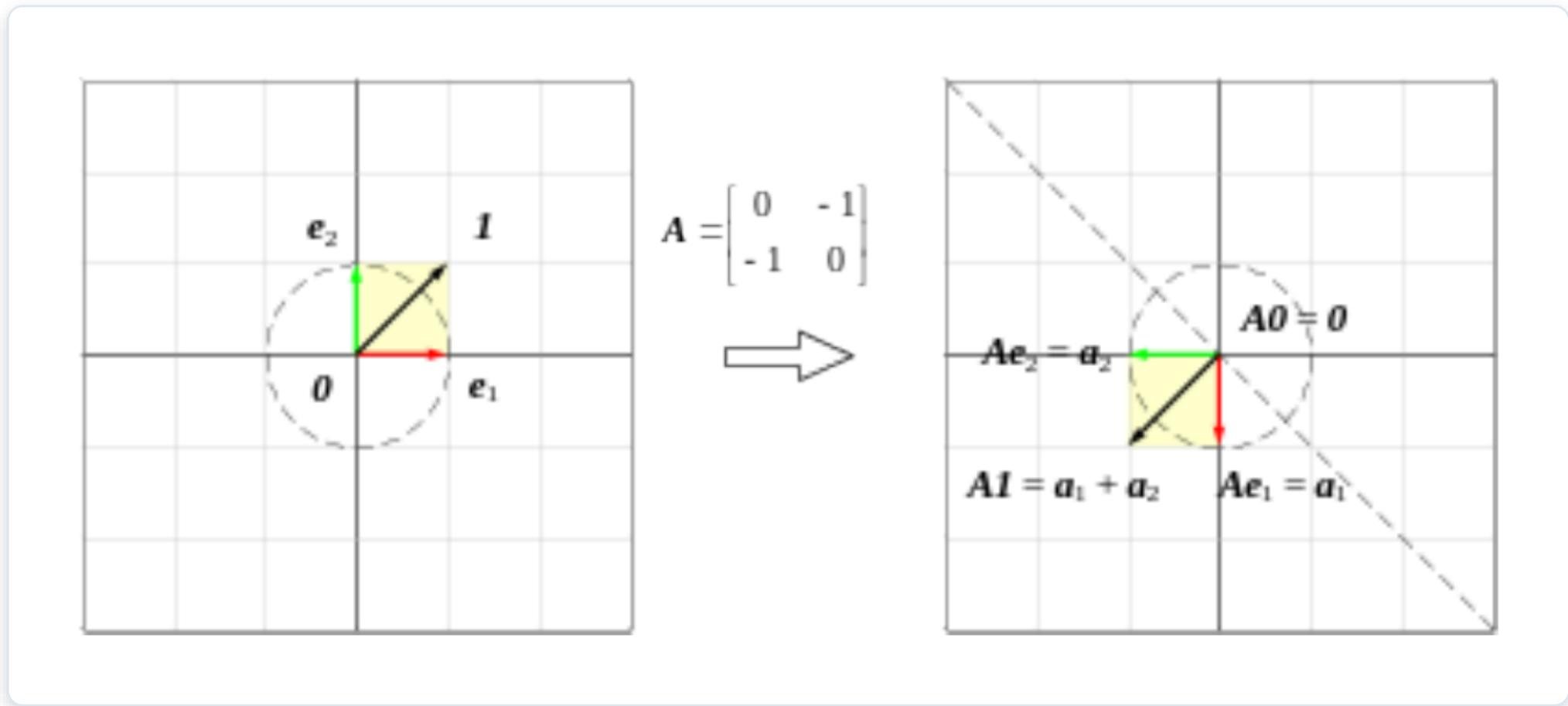


Fig 9 · Matrix A Is 2×2 , Reflection

- a_1, a_2 are unit vectors and orthogonal, but this orthogonal matrix differs from Fig 6's rotation — determinant -1



Custom Function: Grid Data

grid(A): apply a linear transform A to every point on an integer grid

- `numpy.arange()` creates the integer grid points on the horizontal and vertical axes
- `numpy.meshgrid()` expands the two 1-D arrays into a 2-D grid: `xx1` holds all x-coords, `xx2` all y-coords
- `numpy.stack()` stacks the two grids into a 3-D array, convenient for the matrix operation that follows
- `@` is matrix multiplication; `A.T` is **A**'s transpose, needed to match the grid data's shape

```
## 初始化
import numpy as np
import matplotlib.pyplot as plt

## 自定义函数, 网格数据
def grid(A):

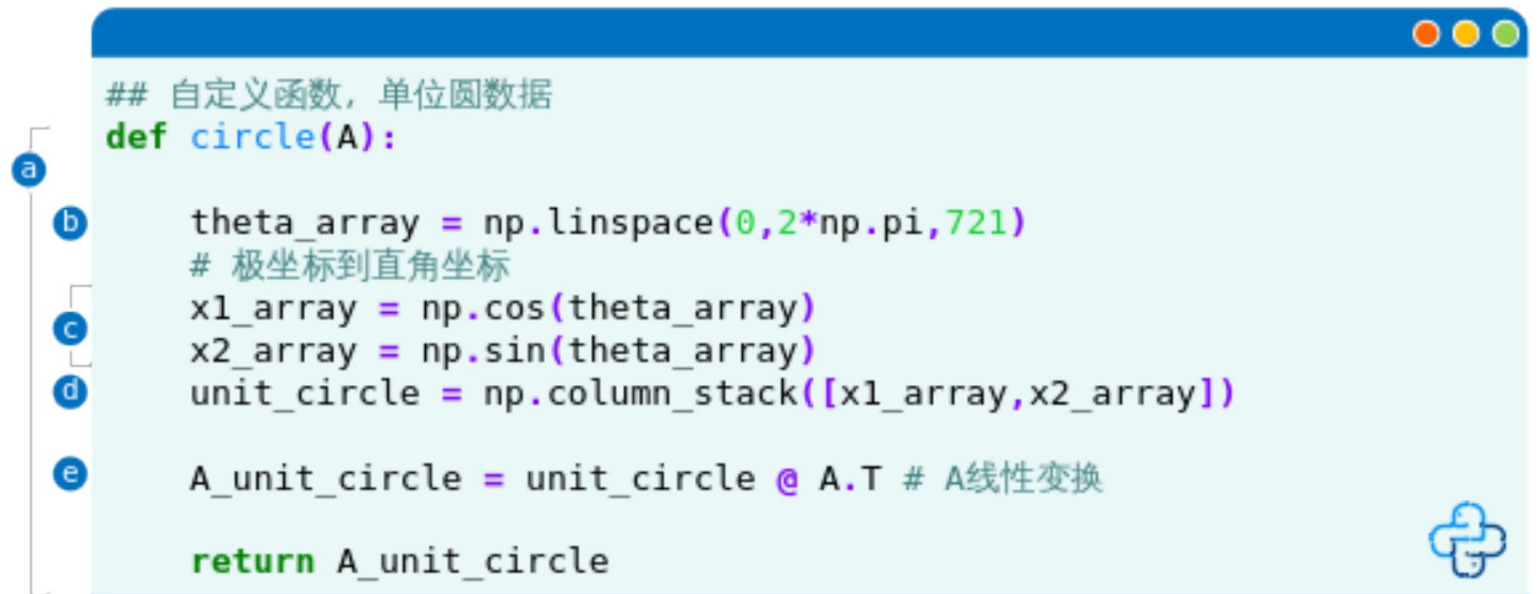
    x1_grids = np.arange(-10,10+1) # 横轴网格点
    x2_grids = np.arange(-10,10+1) # 纵轴网格点
    xx1, xx2 = np.meshgrid(x1_grids, x2_grids) # 生成网格
    X_grid = np.stack([xx1, xx2], axis=2) # 将网格堆叠为3维数组
    A_X_grid = X_grid @ A.T # A线性变换

    return A_X_grid
```

Custom Function: Unit Circle Data

circle(A): generate unit-circle data and apply a linear transform

- `numpy.linspace()` creates 721 angle points between 0 and 2π
- With radius 1 by default, compute cosine (x-coord) and sine (y-coord) for each angle
- `numpy.column_stack()` merges the coordinates into a 2-D array of points on the unit circle
- Transform every point on the unit circle; this example produces a rotated ellipse



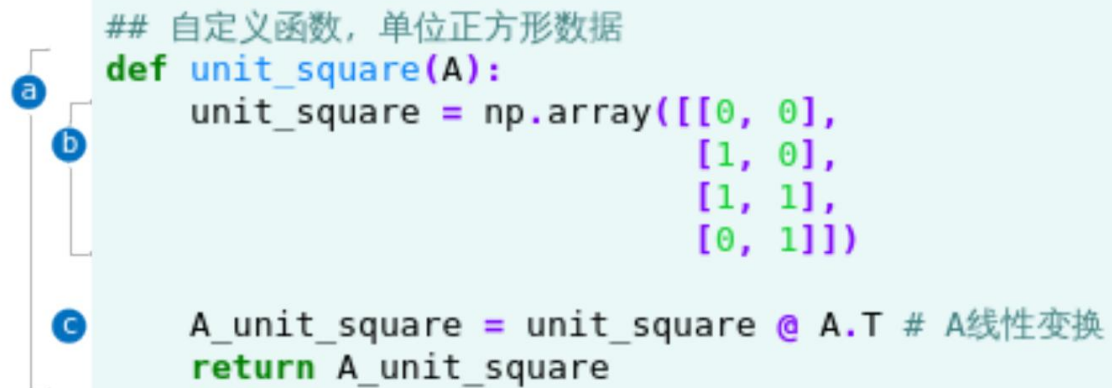
```
## 自定义函数, 单位圆数据
def circle(A):
    a
    b    theta_array = np.linspace(0, 2*np.pi, 721)
        # 极坐标到直角坐标
    c    x1_array = np.cos(theta_array)
    d    x2_array = np.sin(theta_array)
        unit_circle = np.column_stack([x1_array, x2_array])
    e    A_unit_circle = unit_circle @ A.T # A线性变换

    return A_unit_circle
```

Custom Function: Unit Square Data

unit_square(A): generate the unit square's four vertices and transform them

- The four row vectors $[0,0]$, $[1,0]$, $[1,1]$, $[0,1]$ are the unit square's four vertices
- $A.T$ again transforms the four vertices, giving the transformed quadrilateral's coordinates



```
## 自定义函数，单位正方形数据
def unit_square(A):
    unit_square = np.array([[0, 0],
                             [1, 0],
                             [1, 1],
                             [0, 1]])

    A_unit_square = unit_square @ A.T # A线性变换
    return A_unit_square
```

Custom Function: Three Special Column Vectors

vectors(A): compute the transformed e_1 , e_2 , and all-ones vector $\mathbf{1}$

- Define $\mathbf{e}_1 = [1, 0]^T$ and transform it with $A @ \mathbf{e}_1$
- Define $\mathbf{e}_2 = [0, 1]^T$ and apply the same transform with $A @ \mathbf{e}_2$
- Add $\mathbf{e}_1$ and $\mathbf{e}_2$ to get the all-ones vector $\mathbf{1}$, then transform it too



```
## 自定义函数, 三个列向量
def vectors(A):
    e1 = np.array([[1],
                  [0]])
    Ae1 = A@e1 # A线性变换

    e2 = np.array([[0],
                  [1]])
    Ae2 = A@e2 # A线性变换

    all_1 = e1 + e2
    A_all_1 = A@all_1 # A线性变换

    return Ae1, Ae2, A_all_1
```

Custom Visualization Function

visualize(A): plot the grid, unit square, unit circle, and three vectors together

- Call the four helper functions to build the transformed grid, unit square, unit circle, and three vectors
- Draw the transformed grid row by row, then column by column; quiver() draws the three arrows, plot() the circle
- fill() draws the transformed unit square, shaded pale yellow, to make the parallelogram clear at a glance

```
## 定义可视化函数
def visualize(A):
    fig, ax = plt.subplots()

    # 调用自定义函数创建数据
    A_X_grid = grid(A)
    A_unit_square = unit_square(A)
    A_unit_circle = circle(A)
    Ae1, Ae2, A_all_1 = vectors(A)

    # 绘制变换后的网格
    for i in range(A_X_grid.shape[0]):
        plt.plot(A_X_grid[i, :, 0], A_X_grid[i, :, 1],
                 'k-', linewidth=0.5) # 绘制网格行
    for j in range(A_X_grid.shape[1]):
        plt.plot(A_X_grid[:, j, 0], A_X_grid[:, j, 1],
                 'k-', linewidth=0.5) # 绘制网格列

    # 绘制变换后的列向量
    plt.quiver(0, 0, Ae1[0], Ae1[1],
               angles='xy', scale_units='xy',
               scale=1, color=[1, 0, 0], zorder = 1e5)
    plt.quiver(0, 0, Ae2[0], Ae2[1],
               angles='xy', scale_units='xy',
               scale=1, color=[0, 1, 0], zorder = 1e5)
    plt.quiver(0, 0, A_all_1[0], A_all_1[1],
               angles='xy', scale_units='xy',
               scale=1, color=[0, 0, 0], zorder = 1e5)

    # 绘制变换后单位圆
    plt.plot(A_unit_circle[:,0], A_unit_circle[:,1],
             c = 'k', ls = '--')

    # 绘制变换后单位正方形
    plt.fill(A_unit_square[:,0], A_unit_square[:,1],
             c = '#FFFFCC', ls = '--', zorder = 1)

    # 装饰
    plt.axvline(x=0, color='k', zorder=0)
    plt.axhline(y=0, color='k', zorder=0)
    ax.tick_params(axis='both', which='both', length=0,
                  labelbottom=False, labelleft=False)
    ax.set_aspect(1)
    lim = 3; ax.set_xlim([-lim, lim]); ax.set_ylim([-lim, lim])
    plt.xticks(np.arange(-lim, lim))
    plt.yticks(np.arange(-lim, lim))
    ax.grid(linestyle='--', linewidth=0.15, color=[0.8, 0.8, 0.8])
```

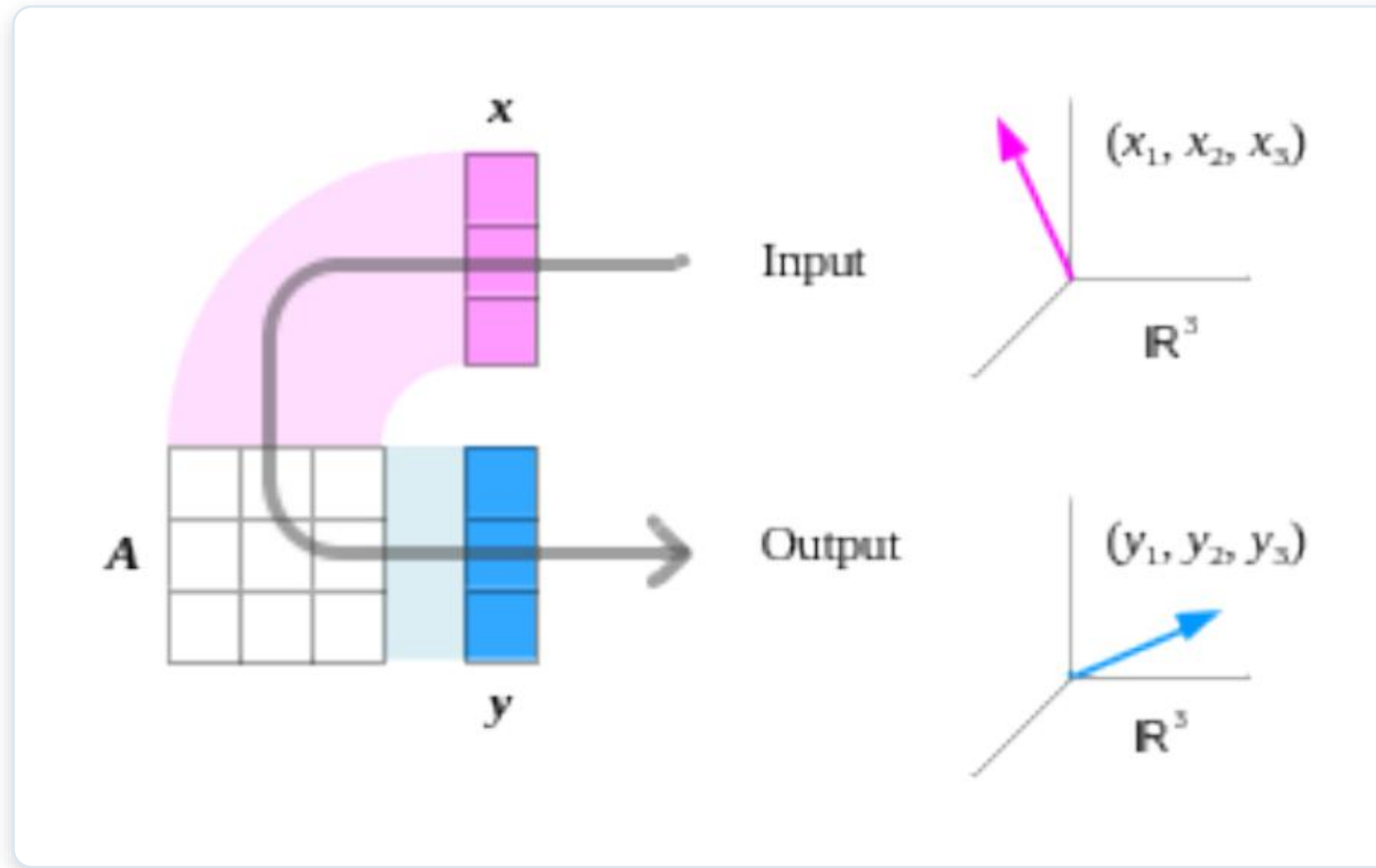
Extending the Linear Transform to Three Dimensions

$$\mathbf{A}\mathbf{x} = \mathbf{y}$$

- A 3×3 matrix $\mathbf{A}$ maps a 3-D vector $\mathbf{x}$ to a 3-D vector $\mathbf{y}$; both $\mathbf{x}$ and $\mathbf{y}$ are points in 3-D space
- In (3×3)@(3×1), the shared 3 is “cancelled”, leaving shape (3×1) — exactly as in the 2×2 case

Fig 10 · Matrix A Is 3×3

- Both input $\mathbf{x}$ and output $\mathbf{y}$ are vectors in 3-D space starting at the origin



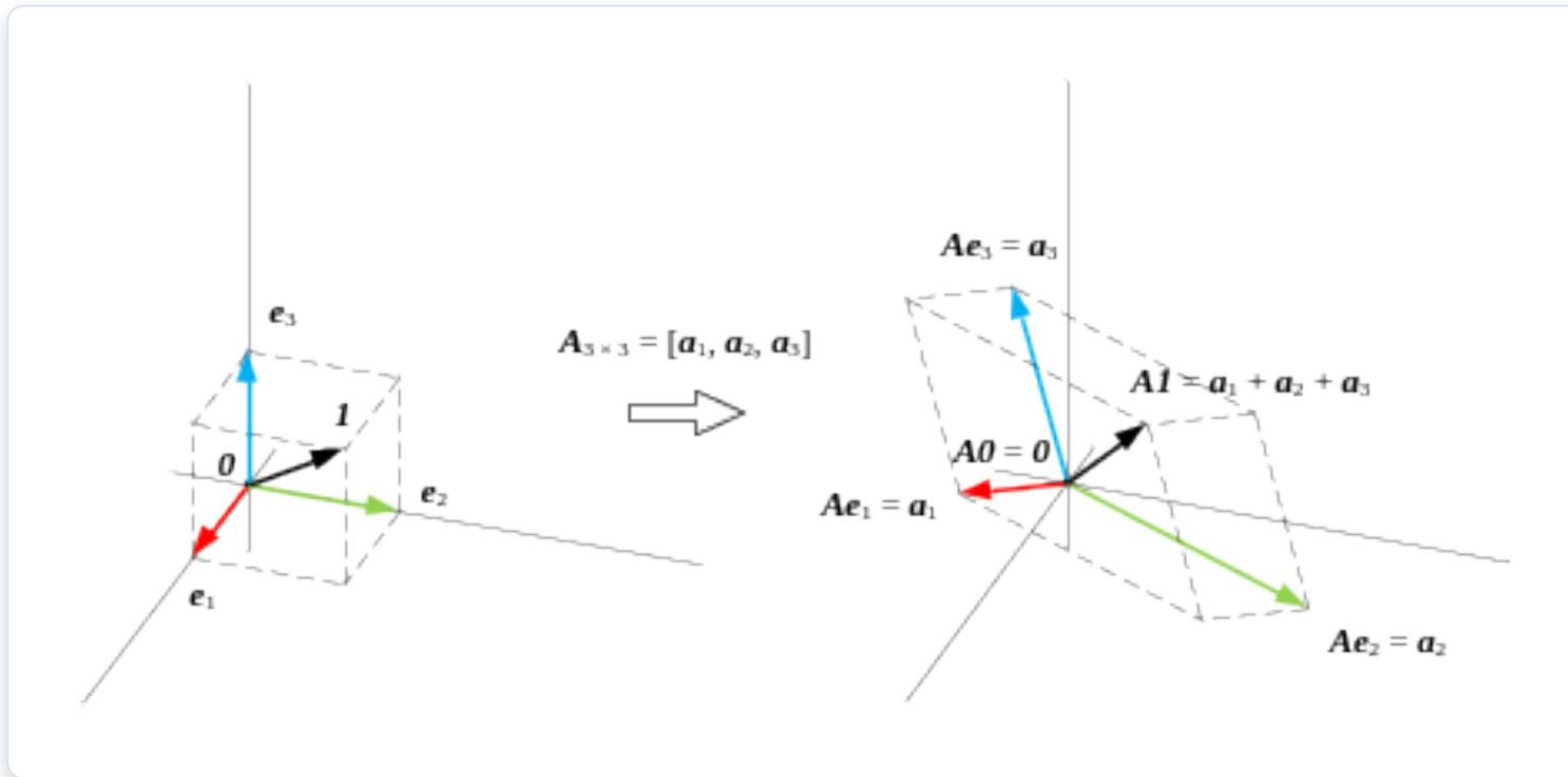
$$Ae_1=a_1, Ae_2=a_2, Ae_3=a_3, A1=a_1+a_2+a_3$$

$$Ae_1=a_1 \quad Ae_2=a_2 \quad Ae_3=a_3 \quad A1=a_1+a_2+a_3$$

- Again the origin stays fixed in 3-D; Ae_1, Ae_2, Ae_3 extract A 's three columns
- The unit cube becomes a parallelepiped: if a_1, a_2, a_3 are independent, the grid spans basis $[a_1, a_2, a_3]$

Fig 11 · Special Column Vectors in Three Dimensions

- Under the linear transform, the unit-cube grid becomes a parallelepiped grid: parallel and equidistant, origin fixed



SUMMARY

What Did Matrix Multiplication Show Us About Transforms?

Special vectors: $\mathbf{0} \rightarrow \mathbf{0}$ (origin fixed), $\mathbf{e}_1, \mathbf{e}_2, \mathbf{e}_3 \rightarrow$ the matrix's columns, $\mathbf{1} \rightarrow$ their sum

Grid transform: unit square/cube $\rightarrow$ parallelogram/parallelepiped, staying parallel and equidistant

Determinant: $|\det(\mathbf{A})|$ is the area/volume scale factor; the sign reflects the columns' relative orientation

Five transforms: scaling (diagonal), rotation (orthogonal, $\det=1$), shear (triangular, $\det=1$), projection ($\det=0$), reflection (orthogonal, $\det=-1$)

Key Terms in This Section

Linear Mapping 线性映射 — matrix multiplication $\mathbf{Ax}=\mathbf{y}$ maps an input vector to an output vector

Rotation 旋转 — preserves length and angle; orthogonal matrix, determinant 1

Orthogonal Projection 正交投影 — projects onto a subspace, reduces dimension, determinant 0

Orthogonal Matrix 正交矩阵 — a square matrix satisfying $\mathbf{AA}^T=\mathbf{A}^T\mathbf{A}=\mathbf{I}$

Determinant 行列式 — reflects the area/volume scaling and orientation of a transform

Scaling 缩放 — stretches or compresses proportionally; the matrix is diagonal

Shear 剪切 — tilts a shape along one direction; triangular matrix, determinant 1

Reflection 镜像 — flips about a line through the origin; orthogonal matrix, determinant -1

Basis 基底 — a set of independent vectors that spans a space

Parallelepiped 平行六面体 — the 3-D grid cell formed by transforming the unit cube

Extended Reading

Further Reading · 8.2 Scaling

Later in this chapter — diagonal matrices and scaling transforms in detail

Further Reading · 8.3 Rotation

Later in this chapter — rotation matrices, rotation angle, orthogonal matrices

Further Reading · 8.5 Orthogonal Projection

Later in this chapter — orthogonal projection, Gram-Schmidt, QR decomposition

Open-Source Repository

All code and figures for the book: <https://github.com/Visualize-ML>

Q1 – Q8

Q1 Compute how each of these 4 matrices transforms $\mathbf{0}$, $\mathbf{e}_1$, $\mathbf{e}_2$, $\mathbf{1}$:
(a) $\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$ (b) $\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$ (c) $\begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}$ (d) $\begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}$.

Q3 Which matrices in Q1 have linearly dependent columns?

Q5 Compute the length (L2 norm) of each column vector in Q1, and the angle between the columns (via inner product).

Q7 Compute the determinant of each matrix in Q1; find the quickest way, and explain the sign of each result.

Q2 Describe in words what geometric transform each matrix in Q1 performs; judge first, then verify visually with this section's code.

Q4 Which matrices in Q1 have columns that form a basis? Identify the basis type and explain why.

Q6 On grid paper, draw the parallelogram formed by each matrix's column vectors in Q1.

Q8 For each matrix in Q1, decide whether an inverse exists; if so, compute it.

Q9 – Q16

Q9 Write a Python function `transform_square(A)` that returns the transformed coordinates of the unit square's four vertices for a 2×2 matrix A .

Q11 Use `numpy.linalg.det()` to verify: a rotation matrix always has determinant 1, and a reflection matrix always has determinant -1 .

Q13 This section's code uses $A.T$ on the grid data — explain why $A.T$ is needed instead of A (hint: the grid data's shape).

Q15 For $A = \begin{bmatrix} 3 & 0 \\ 0 & 0.5 \end{bmatrix}$, decide whether it is orthogonal, and compute how it transforms the unit circle.

Q10 Write a Python function `is_orthogonal(A)` that checks whether a square matrix A satisfies $AA^T = A^T A = I$.

Q12 Build a 3×3 permutation matrix, compute how it transforms e_1 , e_2 , e_3 , and the all-ones vector, and explain the geometric meaning.

Q14 Write a Python function `grid_area(A)` that uses the determinant to compute the area of the parallelogram formed from the unit square.

Q16 Construct a singular (non-invertible) matrix and verify it collapses the unit-square grid onto a single line (dimension drop).

Q17 – Q25

Q17 With matplotlib, plot how the scaling matrix $\text{diag}(2, 0.5)$ and a 30-degree rotation matrix each transform the unit circle, and compare.

Q19 For the 3×3 matrix $A = \text{diag}(1, 1, -1)$, compute its transform of e_1 , e_2 , e_3 , and the all-ones vector, and name the transform.

Q21 Self-study: use numpy to find the eigenvalues and eigenvectors of a 2×2 matrix, and relate them to this section's scaling and rotation.

Q23 Write a Python function `compose(*matrices)` that returns the product of several matrices in order, and verify $(AB)^T = B^T A^T$.

Q25 Design a 3×3 matrix that combines both scaling and shear, and verify it visually with code.

Q18 Prove that $A0=0$ holds for any square matrix A (hint: use the linearity of matrix multiplication).

Q20 Write a Python function `vectors_3d(A)`, modeled on Code 4, that returns the transformed e_1 , e_2 , e_3 for a 3×3 matrix A .

Q22 Modify Code 5's `visualize` function so the axis limits scale automatically with the transform, instead of a fixed $[-3, 3]$.

Q24 Pick any 2×2 matrix A ; compare how A and A^T each transform the same unit-square grid — are the results the same? Why?

10 Prompts to Explore with AI

- 1 Explain why a matrix $\mathbf{A}$ always maps the zero vector $\mathbf{0}$ to $\mathbf{0}$, and why that matters for testing linearity.
- 2 Prove $(\mathbf{AB})^T = \mathbf{B}^T \mathbf{A}^T$, and explain how it helps in understanding a chain of composed transforms.
- 3 Use Python to transform the same unit square with a scaling, a rotation, and a shear matrix, and compare the resulting grids.
- 4 Explain why an orthogonal matrix must have unit-length, mutually orthogonal columns, and how this relates to rotation and reflection.
- 5 Explain how the sign of a determinant reflects the relative orientation of two column vectors.

10 Prompts to Explore with AI

6 Explain why orthogonal projection reduces dimension, and what a determinant of 0 means here.

7 Use Python to verify that a 3×3 matrix $\mathbf{A}$ transforms the unit cube into a parallelepiped whose volume equals $|\det(\mathbf{A})|$.

8 Explain the relationship between a matrix $\mathbf{A}$'s columns being linearly independent and whether they can form a basis.

9 Design an experiment comparing how matrix $\mathbf{A}$ and its transpose $\mathbf{A}^T$ each transform the same unit-square grid.

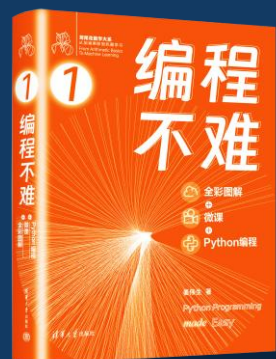
10 Explain why this chapter calls geometric transforms “the foundation for the matrix decompositions later in this book”, with an example.

数学

数学基础

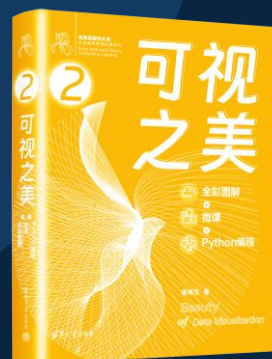
线性代数

概率统计

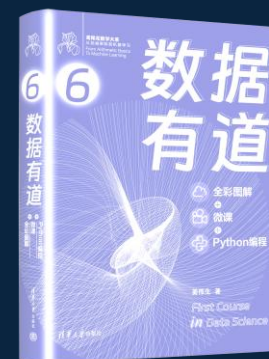
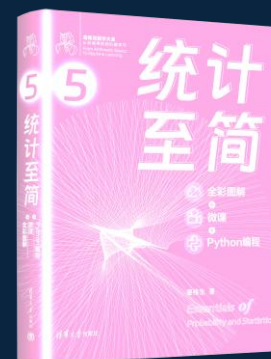
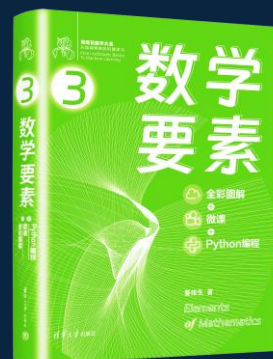


Python编程

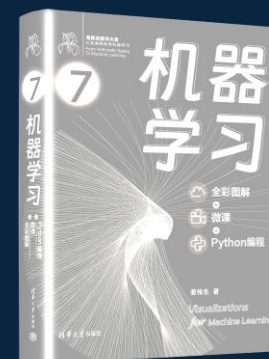
工具



数据可视化



数据分析
图和网络



回归、降维
分类、聚类

实践

